

Problem - 0

```
def fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
```

Recursive calls for fib(5)

fib(5)

fib(4)

fib(3)

fib(2)

fib(1) → returns 1

fib(0) → returns 0

fib(2) returns 1

fib(1) → returns 1

fib(3) returns 2

fib(2)

fib(1) → returns 1

fib(0) → returns 0

fib(2) returns 1

fib(4) returns 3

fib(5) returns 5

Problem - 1

2) Time Complexity:

we insert the first element of each array into the heap. Since there are k arrays, this takes $O(k \log k)$ time

- each time we extract the smallest element from heap, it takes $O(\log k)$

after extraction the next element from the corresponding array which also takes $O(\log k)$

Since $k \times N$ elements process, we perform $k \times N$ heap operation

the total time complexity is $O(N k \log k)$

ways to improve the implementation:

- * Instead of using a heap, we can use merge two sorted arrays at a time.
- * Parallel processing:
If N is large, we can use multithreading to merge different parts in parallel and then merge the results.

Problem-2.

2) Time complexity Analysis:

1) N is the size of the array

1) Traversing the array once using a single loop $\rightarrow O(N)$

2) updating elements in-place $\rightarrow O(1)$ per operation

total time complexity : $O(N)$

3) ways to improve the implementation:

* Using `Set()`: to remove duplicates would work but takes $O(N)$ extra space, which is extra. Since we can solve by $O(1)$